



Samueli
School of Engineering

CS-M151B

Computer Systems Architecture

Blaise Tine
UCLA Computer Science

Attendance



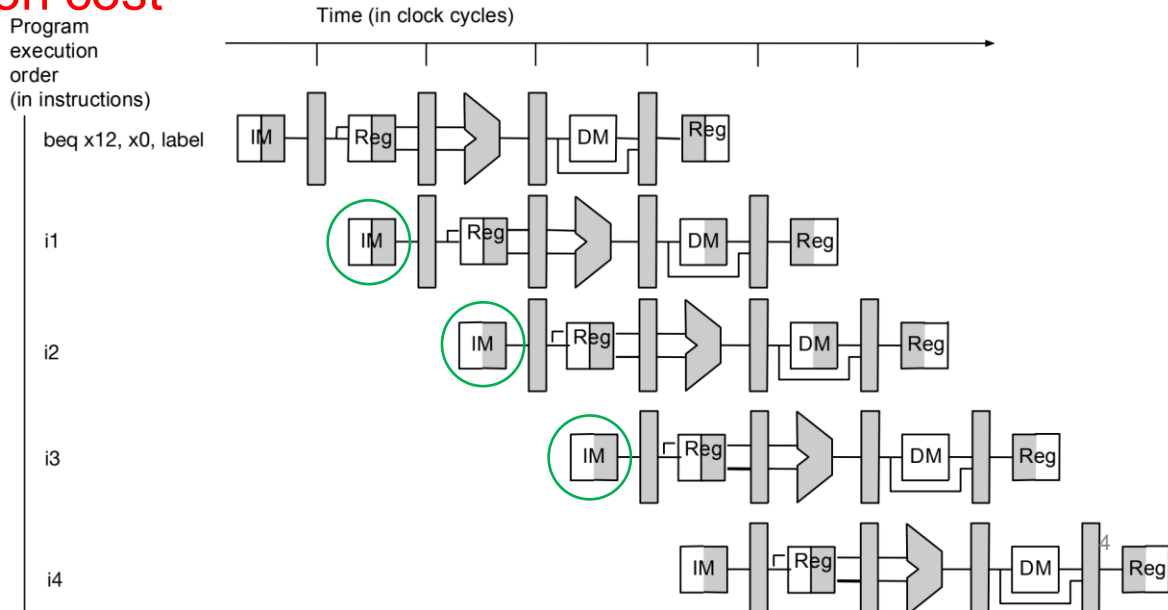
Recap: Control Hazards

RISC-V branches

- Conditional branches
 - `BEQ r1, r2, imm;    // if (r1 == r2) => PC = PC + imm`
- Unconditional branches
 - `JAL rd, imm;        // rd = PC + 4; PC = PC + imm`
 - `JALR rd, rb, imm;   // rd = PC + 4; PC = rb + imm`

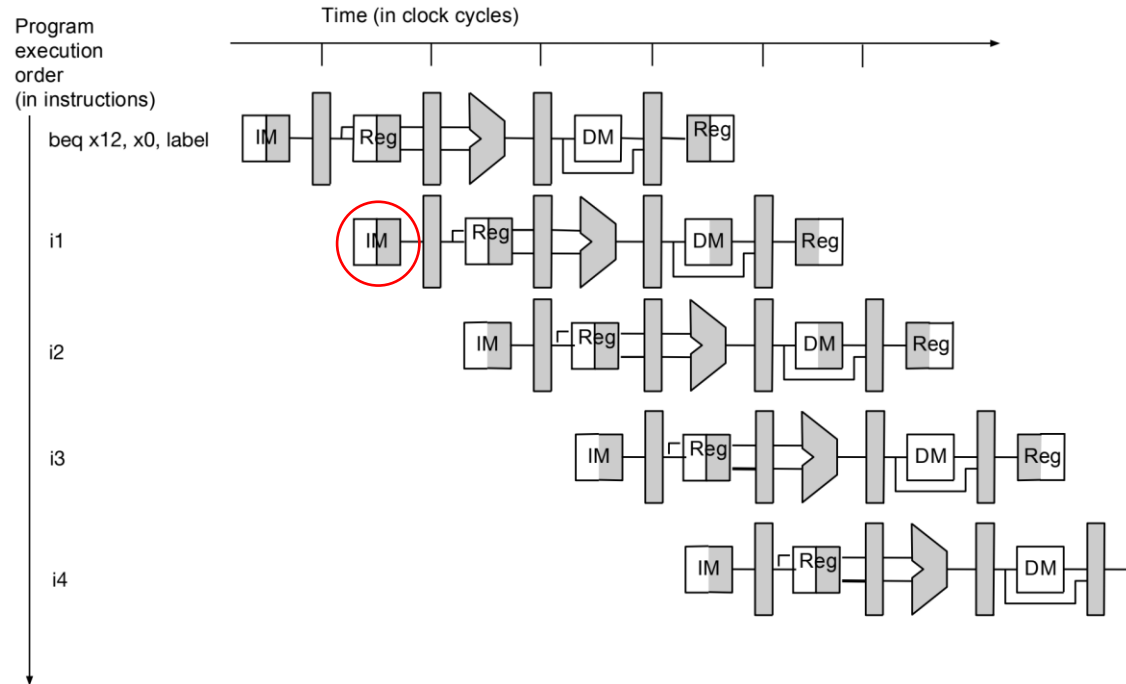
Recap: Control Hazards: Speculation

- Predict: Not taken
 - Emulate delay branch
 - Pros: no wasted cycles
 - Cons: mis-prediction cost



Control Hazards: Speculation

- Predict: Always Taken
- What we need
 - Target?
 - Condition?



Recap: Control Hazards: Speculation

- Predicting **branch target** using **BTB**

- PC address is 4-byte aligned

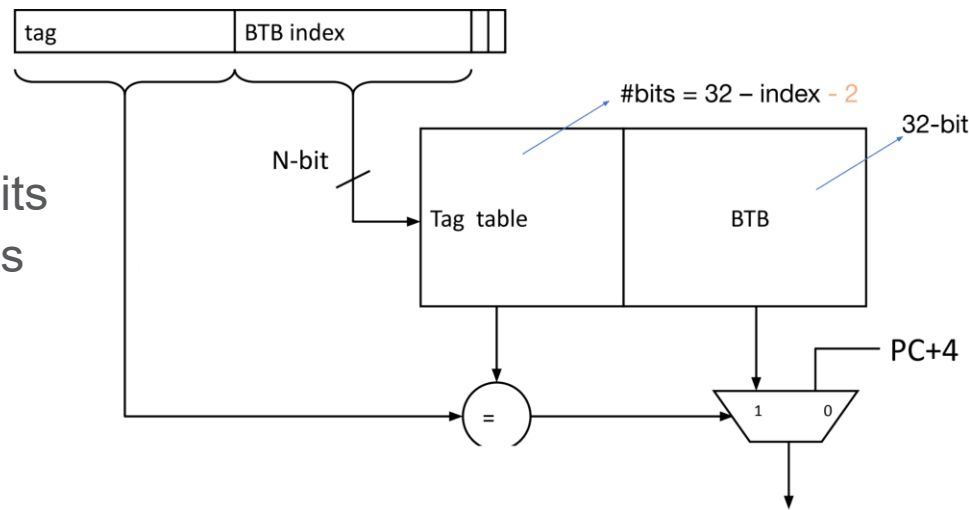
lower 2 bits are zero

- Formulas:

$\text{index_bits} = \log_2(\text{btb_size})$

$\text{pc_index} = (\text{pc} \gg 2) \& \text{index_bits}$

$\text{pc_tag} = (\text{pc} \gg 2) \gg \text{index_bits}$

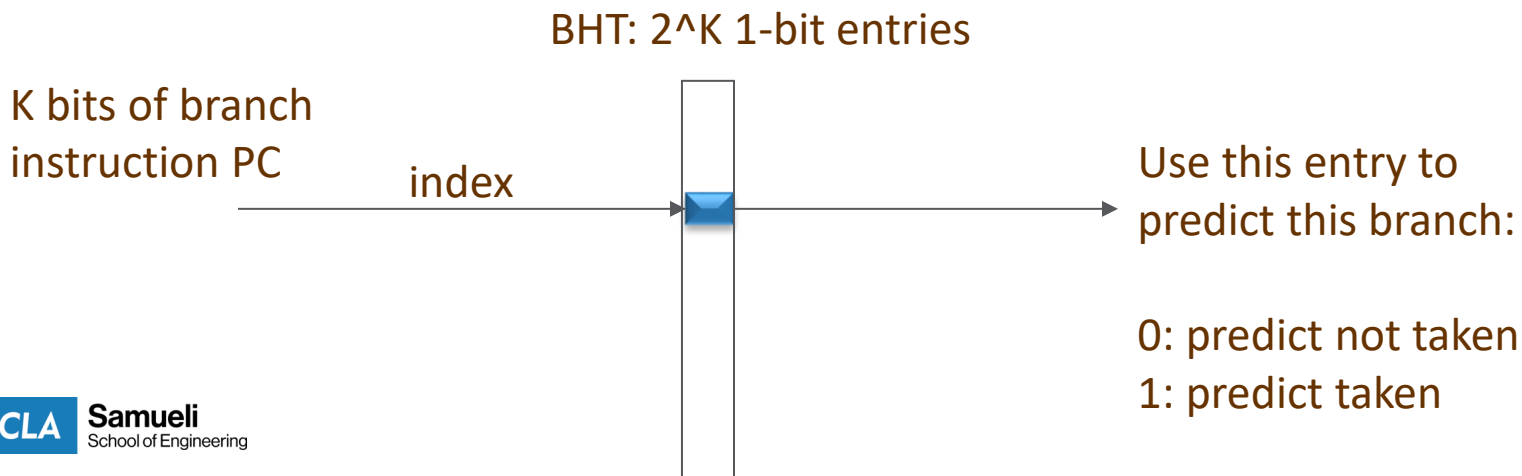


Recap: Control Hazards: Speculation

- Predicting **branch condition** using a **branch predictor**
 - Predict the outcome dynamically using the branch history
 - Recurring branches tend to repeat their pattern (e.g. loops)

Recap: Control Hazards: Speculation

- One-Bit Branch Predictor
 - BTB entry hold a single bit: taken=1 or not_taken=0.
 - Index the table using lower K bits of PC
 - Update the table when the condition is resolved.



Control Hazards: Speculation

- One-Bit Branch Predictor
 - Examples

DC08: TTTTTTTTTT ... TTTTTTTTTNTTTTTTTTTT...

← 100,000 iterations →

How often is branch outcome != previous outcome?

2 / 100,000

99.9%

DC44: TTTTTT ... TNTTTTTT ... TNTTTTTT ...

← 2 / 100 →

2 / 100

98.0%

DC50: TNTNTNTNTNTNTNTNTNTNTNTNTNTNTNT...

2 / 2

2 / 2

00.0%

Control Hazards: Speculation

- One-Bit Is not enough!

Example: short loop (8 iterations)

- Taken 7 times, then not taken once
- Act: TTTTTTTNTTTTTTNTTTTTTTNT...
- Pred: XTTTTTTT**T**NTTTTTT**T**NTTTTTTT**T**N
- Corr: Xoooooooo**MM**oooooooo**MM**oooooooo**MM**
- Misprediction rate: $2/8 = 25\%$

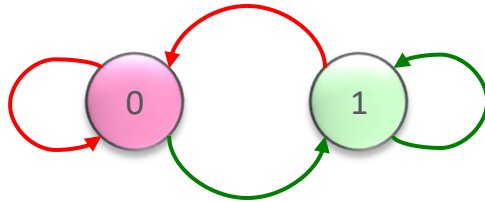
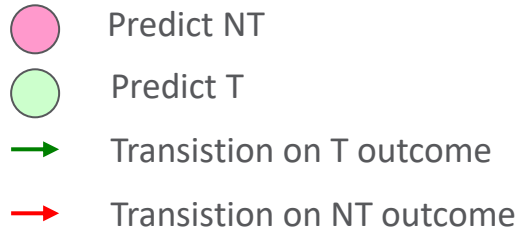
Execute the same loop again

- First always mispredicted (previous outcome was not taken)
- Then 6 predicted correctly
- Then last one mispredicted again

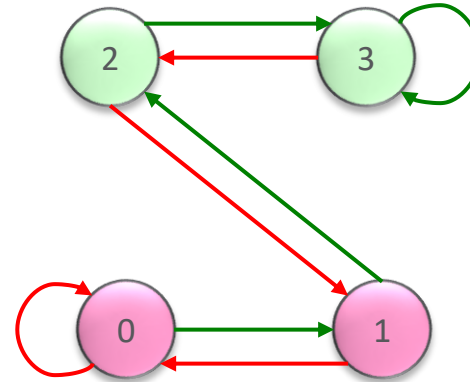
Each fluke/anomaly results in two mispredicts per loop

Control Hazards: Speculation

- Two-Bit Branch Predictor



FSM for Last-time
Prediction



FSM for 2bC
(2-bit Counter)

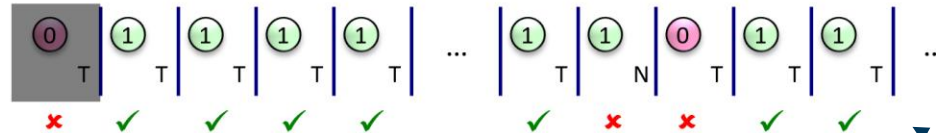
Smith, ISCA 1981

Control Hazards: Speculation

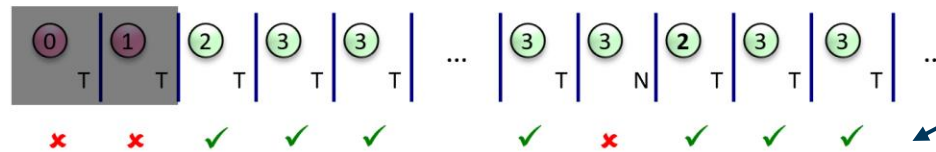
- Two-Bit Branch Predictor

Initial Training/Warm-up

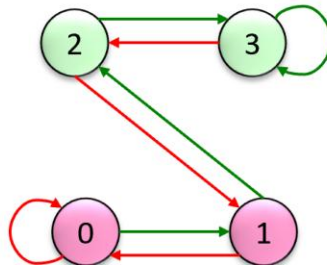
1bC:



2bC:



Prediction result



Only 1 Mispredict per N branches now!

DC08: 99.999%

DC44: 99.0%

DC50: 50.0%

Control Hazards: Speculation

- Can we do better?
 - 1-BC and 2-BC predictors use last branch outcome to predict
 - Observation1: global branch correlation
 - The branch outcome is correlated with outcome from previous branches
 - Observation2: local branch correlation
 - The branch outcome is correlated with prior-outcome from same branch

Control Hazards: Speculation

Global branch correlation

```
if (cond1) {  
    --/--  
}  
if (cond1 and cond2) {  
    --/--  
}
```

Local branch correlation

```
for (x: 0 -> 1023) {  
    --/--  
}
```

Whiteboard

memcpy implementation in C++

```
void memcpy(void *dest, const void *src, size_t n) {  
    unsigned char *d = (unsigned char *)dest;  
    const unsigned char *s = (const unsigned char *)src;  
    for (size_t i = 0; i < n; i++) {  
        d[i] = s[i];  
    }  
}
```

Whiteboard

memcpy implementation in RISC-V assembly

a0 = dest, a1 = src, a2 = n.

memcpy:

beqz a2, end # If length is 0, jump to end

loop:

lbu t0, 0(a1) # Load byte unsigned from src (a1) into temp t0

sb t0, 0(a0) # Store byte from t0 into dest (a0)

addi a0, a0, 1 # Increment dest pointer

addi a1, a1, 1 # Increment src pointer

addi a2, a2, -1 # Decrement counter n

bnez a2, loop # If n != 0, repeat loop

end:

ret # Return to caller (void return)

Control Hazards: Speculation

Two-level Branch Predictor

- Level1: history of last N branches
- Level2: last direction the branch took

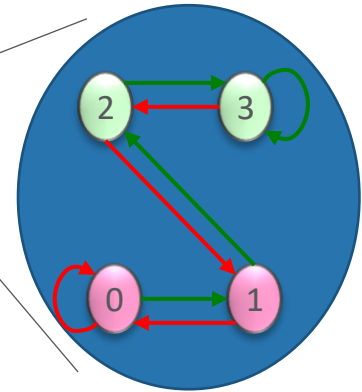
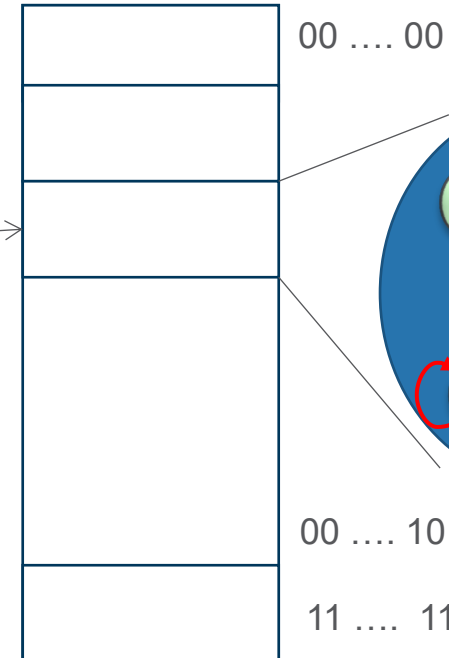
Branch History Register (BHR)



last condition

index

2-Bit BHT



Yeh&patt'92

Control Hazards: Speculation

Two-level Branch Predictor

Initialization value (0 or 1)

Past

Present



← History length →

1 : branch is taken
0: branch is not-taken

New BHR = old BHR<<1 | (br_dir);

Example:

	BHR 00000
Br1 : taken	→ BHR 00001
Br2: not-taken	→ BHR 00010
Br3: taken	→ BHR 00101

Control Hazards: Speculation

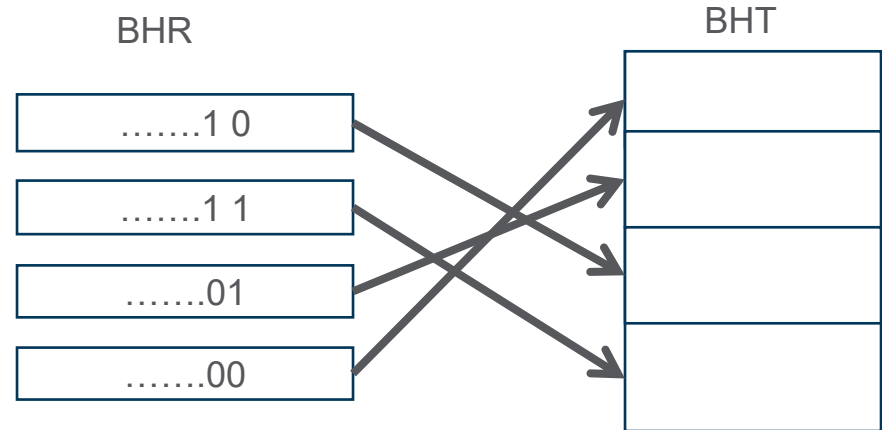
Two-level Branch Predictor

Branch X: if (cond1)

Branch Y: if (cond2)

Branch Z : if (cond1 and cond2)

Branch X	Branch Y	Branch Z
1	0	0
1	1	1
0	1	0
0	0	0



Control Hazards: Speculation

Two-level Branch Predictor Aliasing

Branch A: if (cond1) -> always taken

Branch B: if (cond2) -> always not taken

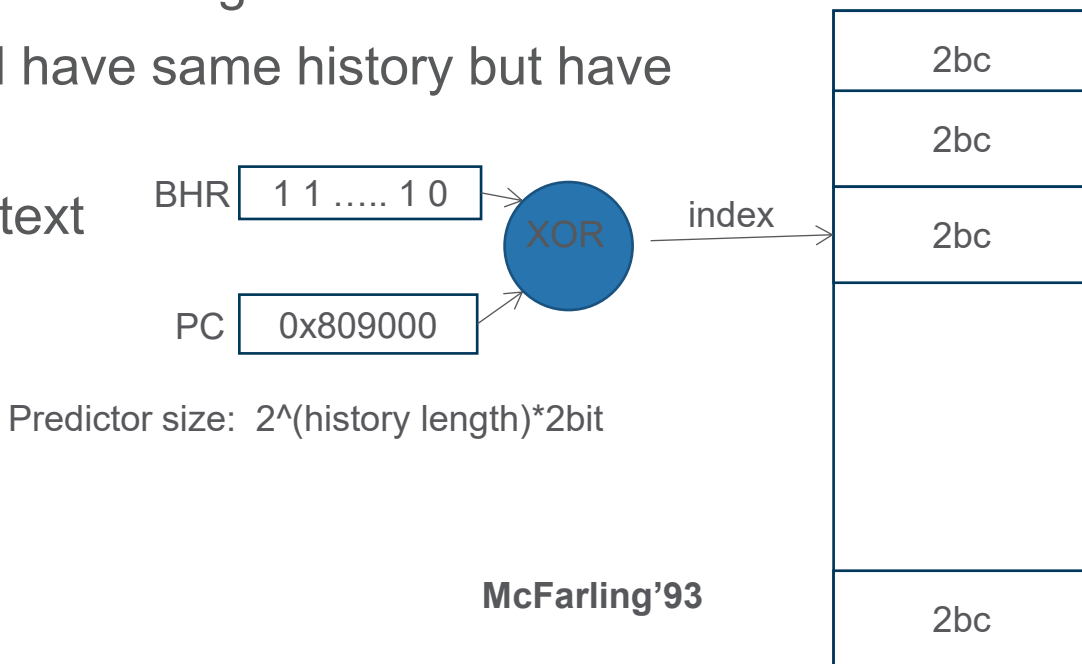
BHR= XX101010 (Taken, No taken)

The 2 branches invalidates each other prediction because they point to same BHT region

Control Hazards: Speculation

Gshare Branch Predictor

- 2-level predictor limitation: Aliasing
- Different branches could have same history but have different outcome
- Need to add branch context



McFarling'93

Control Hazards: Speculation

Gshare Branch Predictor: algorithm

```
predict_func(pc, actual_dir) {  
    index = PC xor BHR  
    taken = (2bit_counters[index] >= 2) ? 1 : 0  
    correctly_predicted = (actual_dir == taken) ? 1 : 0 // stats  
}
```

```
update_func(pc, actual_dir) {  
    index = PC xor BHR  
    if (actual_dir)  
        INC_SAT(2bit_counter[index])  
    else  
        DEC_SAT(2bit_counter[index])  
    BHR = BHR << 1 | actual_dir  
}
```

Control Hazards: Speculation

Improving Return Call Prediction

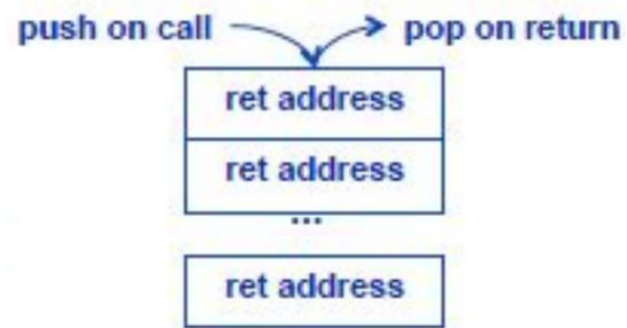
- RISC-V JALR
 - $PC = RF[rs1] + imm$
- Target depends on `rs1`
- Difficult to predict
 - Different callers
 - Nested calls

Control-Flow inst.	Direction at fetch time	Number of possible next fetch addresses?	When is next fetch address resolved?
Conditional	Unknown	2	Decode (<code>rs1 == rs2</code>)
Unconditional	Always taken	1	Decode (<code>PC + offset</code>)
Call	Always taken	1	Decode (<code>PC + offset</code>)
Return	Always taken	Many	Decode (register dependent)
Indirect	Always taken	Many	Decode (register dependent)

Control Hazards: Speculation

Return Stack Buffer (RSB)

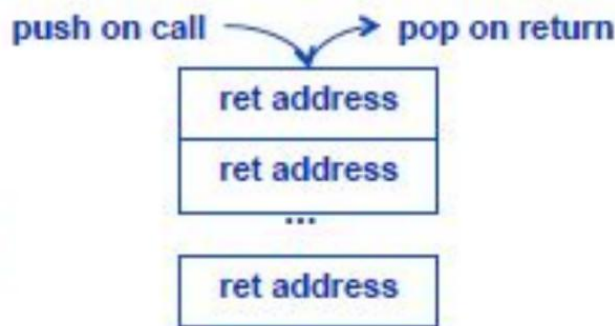
- Use hardware stack
 - About 4-8 entries
- On function call:
 - Push current return address
- On function return:
 - Pop last return address
 - Use that for prediction



Control Hazards: Speculation

Return Stack Buffer (RSB)

- Tail recursion optimization
 - if (stack.top == ra) => skip
 - Avoid polluting stack
- Integration with BTB:
 - Predict NextPC from BTB or RSB
 - How to choose?
 - Extend BTB with “isRA” column



Attendance

- Please Fill the form with today's keyword to register your attendance.



CPU Performance

- How do we improve CPU performance?

$$\text{CPU Time} = \text{IC} * \text{CPI} * \text{CT}$$

IC: instruction count

CPI: cycles per instructions

CT: clock cycle time

Whiteboard: CPU Performance

- Single-cycle vs multi-cycle vs pipelined?

CPU Performance

- Single-cycle architecture Performance
 - Pros: CPI=1
 - Cons: Long CT

$$\text{CPU Time} = \text{IC} * \text{CPI} * \text{CT}$$

IC: instruction count

CPI: cycles per instructions

CT: clock cycle time

CPU Performance

- Multi-cycle architecture Performance
 - Pros: Smaller CT
 - Cons: Longer CPI

$$\text{CPU Time} = \text{IC} * \text{CPI} * \text{CT}$$

IC: instruction count

CPI: cycles per instructions

CT: clock cycle time

CPU Performance

- Pipeline architecture Performance
 - Pros: Smaller CT + Smaller CPI
 - Cons: Hardware cost

$$\text{CPU Time} = \text{IC} * \text{CPI} * \text{CT}$$

IC: instruction count

CPI: cycles per instructions

CT: clock cycle time

CPU Performance

- Downside of more pipeline stages?
 - CT reduction is limited
 - **CPI will increase**: more pipeline hazards
 - Hardware cost: more forwarding paths

$$\text{CPU Time} = \text{IC} * \text{CPI} * \text{CT}$$

IC: instruction count

CPI: cycles per instructions

CT: clock cycle time

CPU Performance

- How do we reduce CPI?
 - Reduce the pipeline stalls
 - Increase the instruction throughput

CPU Performance

- Latency
 - Time it takes to complete one task
- Throughput
 - Number of tasks completed in a given time

CPU Performance

- Example of throughput
 - Computation capacity: FLOPS (floating-point operations per sec)
 - Memory bandwidth: GB/s (memory bytes transfer per sec)

CPU Performance

- IPC: Instructions per cycle
 - Instruction throughput
 - $IPC = 1 / CPI$
 - How many instructions to executed per cycle
 - Higher is better

$$CPU\ Time = (IC * CT) / IPC$$

IC: instruction count

IPC: instructions per cycle

CT: clock cycle time

CPU Performance

- CPU Paralleling Types
 - ILP: instruction-level parallelism
 - TLP: thread-level parallelism
 - MLP: memory-level parallelism

Whiteboard: Super-scalar Pipeline

- Example: $N=2$

SuperScalar Processor

- Exercise: How do we extend scalar pipeline for $IPC=2$?
- IF Stage:
 - PC
 - No replication – why?
 - Instruction Memory
 - Multiple read ports

Nader ECE-M116C

SuperScalar Processor

- Exercise: How do we extend scalar pipeline for IPC=2?
- ID Stage:
 - Decode
 - Replication
 - Register file
 - Multiple read ports

Nader ECE-M116C

SuperScalar Processor

- Exercise: How do we extend scalar pipeline for IPC=2?
- EX Stage:
 - ALU
 - Replication

Nader ECE-M116C

SuperScalar Processor

- Exercise: How do we extend scalar pipeline for IPC=2?
- MEM Stage:
 - Data Memory
 - No replication – why?

Nader ECE-M116C

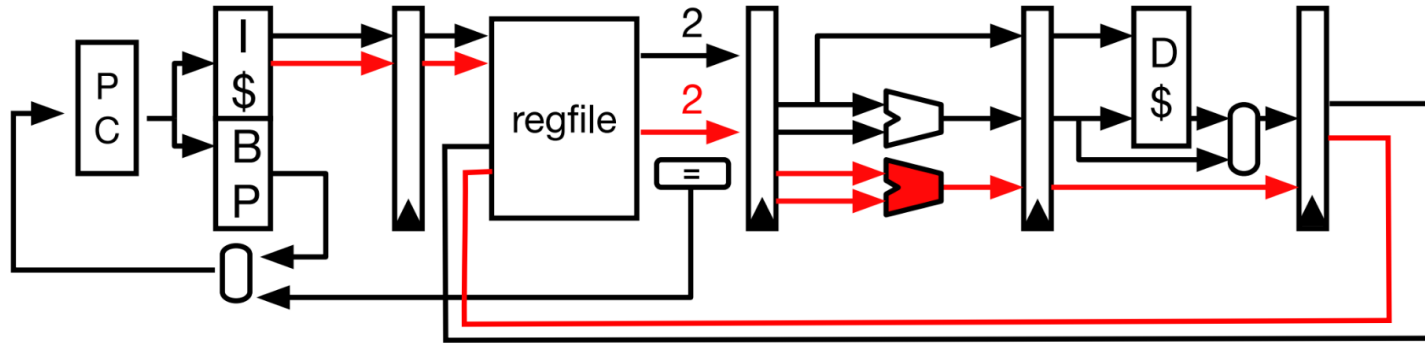
SuperScalar Processor

- Exercise: How do we extend scalar pipeline for $IPC=2$?
- WB Stage:
 - Register File
 - Multiple write ports

Nader ECE-M116C

SuperScalar Processor

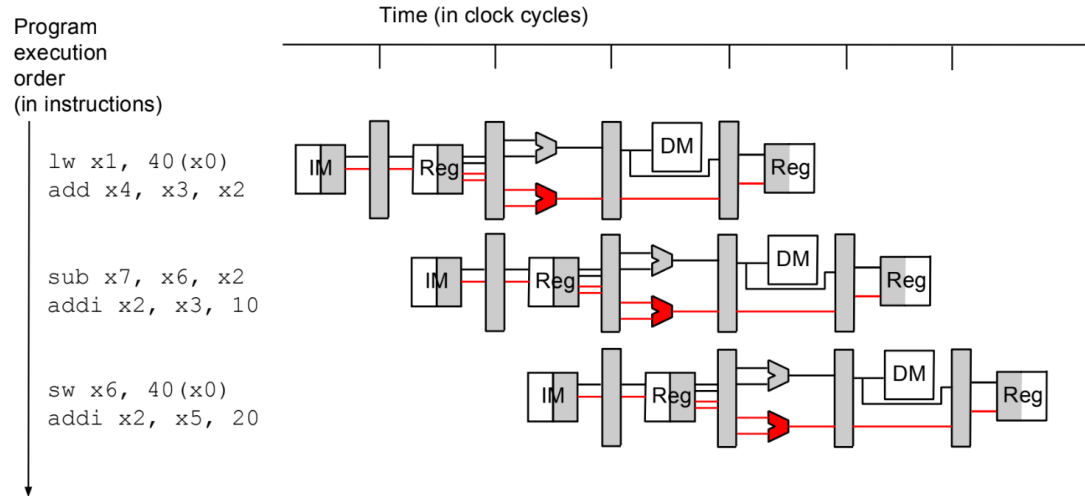
- Exercise: How do we extend scalar pipeline for IPC=2?



Nader ECE-M116C

SuperScalar Processor

- Exercise: How do we extend scalar pipeline for IPC=2?



SuperScalar Processor

- Hardware complexity
 - Replicated stages
 - Data forwarding cost

SuperScalar Processor

- Scheduling complexity?
 - Structural hazards (MEM stage)
 - Data dependency

SuperScalar Processor

- Data dependency issue? (N=2)
 - ADD r1 <- r1, r3
 - ADD r6 <- r4, r3
 - No

SuperScalar Processor

- Data dependency issue? (N=2)
 - ADD r3 <- r1, r2
 - ADD r3 <- r4, r1
 - Yes: WAW: write ordering should be enforced
 - Solution: Delay the second instruction

SuperScalar Processor

- Data dependency issue? (N=2)
 - ADD r3 <- r1, r2
 - ADD r6 <- r4, r3
 - Yes: RAW: r3 read before being written
 - Solution: Delay the second instruction

SuperScalar Processor

- How many RAW checks? (N=4)
 - ADD rd1 <- rs1a, rs1b
 - ADD rd2 <- rs2a, rs2b
 - ADD rd3 <- rs3a, rs3b
 - ADD rd4 <- rs4a, rs4b
 - 12 checks: $6(rs4) + 4(rs3) + 2(rs2)$

SuperScalar Processor

- SuperScalar best-case performance
 - How often instructions have no dependencies?
 - Doesn't happen often enough

SuperScalar Processor

- SuperScalar best-case performance
 - What to do if data dependency check fails?
 - Allow other non-dependent instructions to issue

SuperScalar Processor

- Scalar: can process at most one instruction per cycle
- SuperScalar: can process at most N instructions per cycle

Out of Order Execution

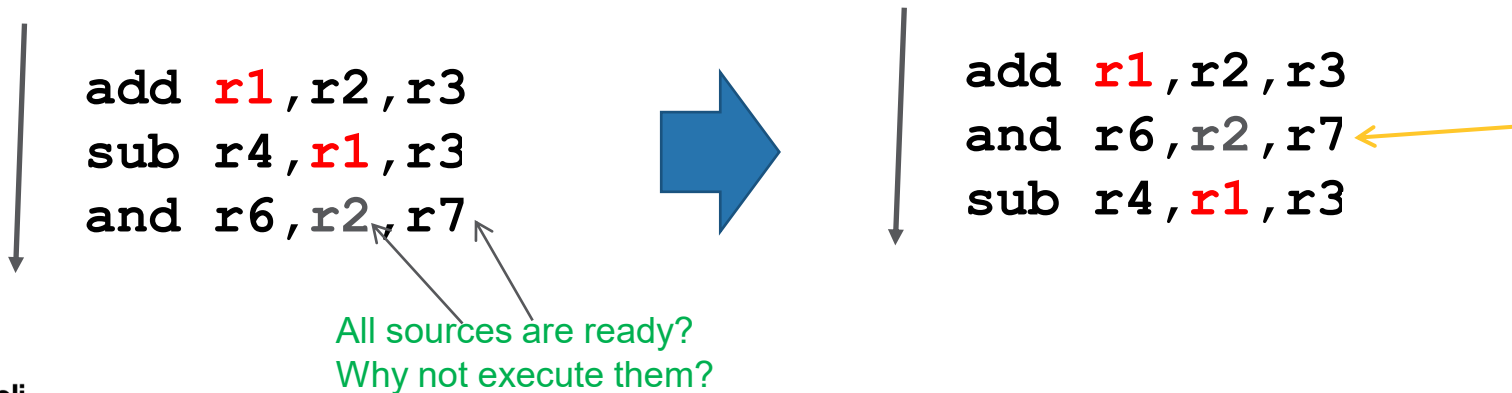
- If the next instruction cannot move forwards due to dependencies, we skip it and pick the next one.
- The instruction window size determines pipeline utilization

Out of Order Execution

- OoO Design Challenges?
 - Finding a ready instruction
 - How do we track dependencies when ordering is broken?
 - How do we handle pipeline hazards?
 - How do we preserve ordering for external states?
 - Registers and memory

Static Scheduling

- Compiler re-order the instruction
 - Internally track dependencies
 - Check register use
- How do we do that in the compiler?



Dynamic Scheduling

- How to reduce register dependencies?

Dynamic Scheduling

- Can we increase the number of registers to work with?
 - You cannot change the ISA (our S/W contract)

Dynamic Scheduling

- **Solution:**
 - Use internal registers: “physical” registers
 - We will call ISA registers “logical” or “architected”
 - initially map logical registers to physical registers

HW Register Renaming

Needed Features:

- mapping mechanism
- physical registers
 - allocation/deallocation mechanism
 - allocated vs. free registers

HW Register Renaming

Exercise:

- Identify all data dependencies

Program code

I1: ADD R1, R3, R3

I2: SUB R2, R1, R5

I3: AND R5, R11, R7

I4: OR R8, R5, R6

I5: XOR R2, R4, R11

RAW 
WAR 
WAW 

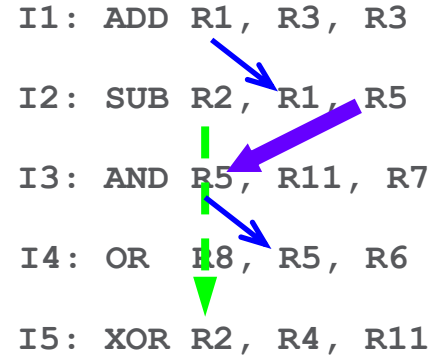
HW Register Renaming

Exercise

- 2 RAWs
- 1 WAR
- 1 WAW
- Which instructions cannot be reordered?

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND R5, R11, R7
I4: OR R8, R5, R6
I5: XOR R2, R4, R11
```



RAW →
WAR →
WAW - - →

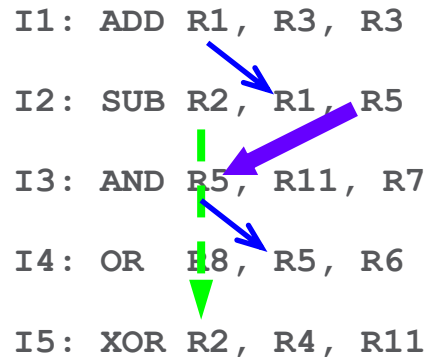
HW Register Renaming

Exercise

- I3 can not exec before I2 because I3 will overwrite R5
- I5 can not go before I2 because it will overwrite R2 with a stale value

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND R5, R11, R7
I4: OR R8, R5, R6
I5: XOR R2, R4, R11
```



RAW →
WAR →
WAW →

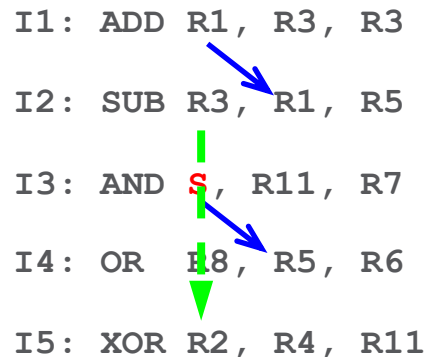
HW Register Renaming

Example

- Let's rename R5 to decouple I2 and I3
- R5 = S
- Removed WAR

Program code

```
I1: ADD R1, R3, R3
I2: SUB R3, R1, R5
I3: AND S, R11, R7
I4: OR R8, R5, R6
I5: XOR R2, R4, R11
```



RAW →
WAR →
WAW - - →

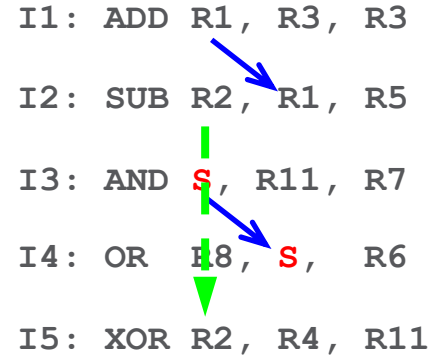
HW Register Renaming

Example

- I4 also needs to change since R5 is S

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND S, R11, R7
I4: OR R8, S, R6
I5: XOR R2, R4, R11
```



RAW 
WAR 
WAW 

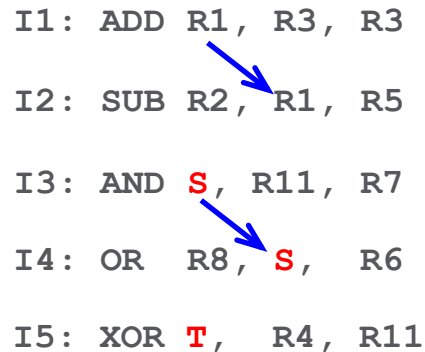
HW Register Renaming

Example

- Let's rename R2 to decouple I2 and I5
- R2 = T
- Removed WAW

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND S, R11, R7
I4: OR R8, S, R6
I5: XOR T, R4, R11
```



RAW 

WAR 

WAW 

HW Register Renaming

Register renaming eliminates

- WAR dependencies
- WAW dependencies
- **We still have RAW dependencies**

Program code

```
I1: ADD R1, R3, R3
      ↓
I2: SUB R2, R1, R5
      ↓
I3: AND S, R11, R7
      ↓
I4: OR  R8, S, R6
      ↓
I5: XOR T, R4, R11
```

RAW →
WAR →
WAW - - →

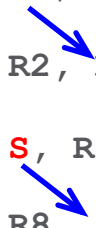
HW Register Renaming

Implementation

- Space for **S**, **T**, etc.
- How do we know when to rename a register?

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND S, R11, R7
I4: OR  R8, S, R6
I5: XOR T, R4, R11
```



RAW 
WAR 
WAW 

HW Register Renaming

Simple Solution

- Do renaming for every instruction
- Change the name of a register each time we decode an instruction that will write to it.
- Remember what name we gave it 😊

Program code

I1: ADD R1, R3, R3

I2: SUB R2, R1, R5

I3: AND S, R11, R7

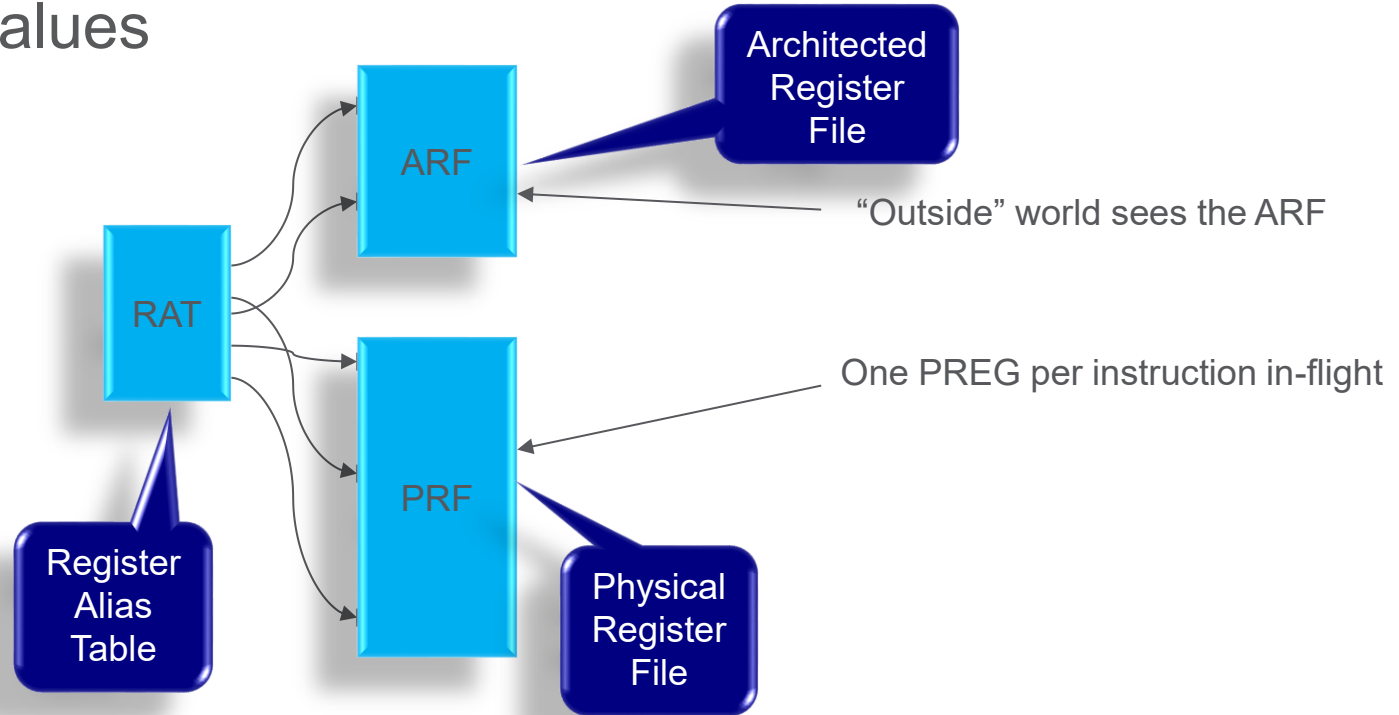
I4: OR R8, S, R6

I5: XOR T, R4, R11

RAW 
WAR 
WAW 

Register File Organization

We need some physical structure to store the register values



Preparing for the midterm

- Covering all lectures until
- All homework assignments
- Project #1
- ISA, CPU pipeline, pipeline hazards, branch predictors
- You will have some design questions
 - Custom ISA / pipeline
 - cost vs perf

Q&A

Acknowledgement

- This course is partly inspired and use materials from the following courses created by my colleagues:
 - ECE-M1126C: Nader Sehatbakhsh @ UCLA
 - CS3220: Hyesoon Kim @ gatech